

TECH-TALKS

Terraform

A Complete Beginner's Textbook

Infrastructure as Code | Cloud Automation | DevOps

From Zero to Hero in Terraform

Version 1.0 | 2024 Edition

Table of Contents

Chapter 1: Terraform Basics	6
1.1 What is Terraform?	6
1.2 Why Terraform?	6
1.3 Infrastructure as Code (IaC)	7
1.4 Declarative vs Imperative	7
1.5 Terraform Use Cases	8
Chapter 2: Terraform Architecture	9
2.1 Terraform CLI	9
2.2 Terraform Core	9
2.3 Providers	9
2.4 Resources	10
2.5 State File	10
Chapter 3: Installation & Setup	11
3.1 Installing on Windows	11
3.2 Installing on macOS	11
3.3 Installing on Linux	11
3.4 Key Setup Commands	12
Chapter 4: Providers	13
4.1 What is a Provider?	13
4.2 Configuring Providers	13
AWS Provider Example	13
Azure Provider Example	14
GCP Provider Example	14
4.3 Multiple Providers	14
4.4 Provider Versioning	14
Chapter 5: Resources	16
5.1 What is a Resource?	16
5.2 Resource Block Syntax	16
5.3 Resource Dependencies	16
Implicit Dependencies (Recommended)	16
Explicit Dependencies (depends_on)	17
Chapter 6: Variables	18
6.1 Input Variables	18
6.2 Variable Types	19

6.3 Using .tfvars Files	19
6.4 Sensitive Variables	19
Chapter 7: Outputs.....	21
7.1 Defining Outputs.....	21
7.2 Viewing Outputs	21
7.3 Outputs in Module Communication	21
Chapter 8: State Management	23
8.1 What is Terraform State?.....	23
8.2 Local vs Remote State.....	23
8.3 State Locking.....	23
8.4 Security Best Practices for State	24
Chapter 9: Backends.....	25
9.1 Local Backend (Default)	25
9.2 S3 Backend (AWS).....	25
9.3 Terraform Cloud Backend.....	25
Chapter 10: Core Terraform Commands	27
10.1 terraform init	27
10.2 terraform plan	27
10.3 terraform apply	28
10.4 terraform destroy	28
10.5 terraform show & terraform state	28
Chapter 11: Modules.....	30
11.1 What is a Module?	30
11.2 Module Structure	30
11.3 Using Modules.....	30
11.4 The Terraform Module Registry	31
Chapter 12: Resource Meta-Arguments.....	32
12.1 depends_on.....	32
12.2 count	32
12.3 for_each	33
12.4 lifecycle.....	33
Chapter 13: Data Sources.....	34
13.1 Resources vs Data Sources	34
13.2 Common Data Source Examples.....	34
Chapter 14: Expressions & Functions	36
14.1 Interpolation Syntax.....	36

14.2 Built-in Functions	36
14.3 Conditional Expressions	37
14.4 For Expressions.....	37
Chapter 15: Workspaces.....	38
15.1 Workspace Concepts.....	38
15.2 Workspace Commands	38
15.3 Using Workspace in Configuration.....	38
Chapter 16: Terraform Import.....	40
16.1 How Import Works	40
16.2 Terraform 1.5+ Generated Config	40
16.3 Import Limitations & Tips	40
Chapter 17: Drift Detection.....	42
17.1 What is Drift?	42
17.2 Detecting Drift.....	42
17.3 Preventing Drift.....	42
Chapter 18: Terraform Cloud & Enterprise	44
18.1 Key Features	44
18.2 Sentinel Policies	44
Chapter 19: Security Best Practices.....	46
19.1 Protecting State Files.....	46
19.2 Handling Sensitive Variables	46
19.3 IAM Least Privilege.....	46
Chapter 20: Testing & Validation.....	48
20.1 Built-in Validation Commands.....	48
20.2 Variable Validation.....	48
20.3 Testing Tools	49
Chapter 21: Best Practices.....	50
21.1 Code Organization.....	50
21.2 The Golden Rules of Terraform	50
21.3 Naming Conventions	51
Chapter 22: Terraform vs Other Tools.....	52
Chapter 23: Terraform in DevOps	53
23.1 GitOps Workflow.....	53
23.2 GitHub Actions CI/CD Example	53
23.3 Kubernetes Integration	54
Chapter 24: Troubleshooting.....	55

24.1 Common Issues & Solutions	55
Init Failures	55
Provider Authentication Errors	55
State Lock Issues.....	56
Dependency Conflicts	56
Common Error Messages	56
Chapter 25: Real-World Use Cases	57
25.1 Provisioning a Complete Web Application Stack on AWS.....	57
25.2 Multi-Cloud Deployment	58
25.3 EKS Kubernetes Cluster Setup.....	58
25.4 Network & Security Automation	59
25.5 Complete Infrastructure Automation Workflow	59
Command Cheat Sheet	60
HCL Quick Reference.....	60

CH 1

Terraform Basics

Understanding what Terraform is and why it matters

Chapter 1: Terraform Basics

Welcome to Terraform! This chapter lays the foundation for everything you'll learn in this textbook. We'll start from scratch — no prior experience required — and by the end of this chapter, you'll understand what Terraform is, why it exists, and how it fits into the modern technology landscape.

1.1 What is Terraform?

Terraform is an open-source Infrastructure as Code (IaC) tool created by HashiCorp. Put simply, it allows you to define, provision, and manage your cloud infrastructure using configuration files — just like writing code.

ANALOGY: The Recipe Book

Terraform is like a recipe book for your infrastructure. Instead of cooking food step-by-step, it 'cooks' your servers, networks, and storage automatically. You write the recipe once (your Terraform config), and it creates the exact same infrastructure every time you run it — no surprises!

Before tools like Terraform existed, setting up infrastructure meant logging into cloud consoles, clicking buttons manually, and hoping you remembered every step. This was error-prone, slow, and impossible to reproduce exactly. Terraform changed all that.

1.2 Why Terraform?

There are many compelling reasons to use Terraform in your projects and organization:

- **Automation** Automation: Eliminate manual setup. Define your infra once and deploy it automatically.
- **Version Control** Version Control: Store Terraform files in Git — track changes, rollback, collaborate with teams.
- **Repeatability** Repeatability: Create identical environments for dev, staging, and production with zero drift.
- **Multi-Cloud** Multi-Cloud Support: Manage AWS, Azure, GCP, and hundreds of other providers from one tool.
- **Speed** Speed: Spin up entire environments in minutes instead of hours or days.
- **Consistency** Consistency: Every deployment follows the same blueprint — no manual errors or forgotten steps.

TIP: Industry Standard

Terraform is used by thousands of companies worldwide — from startups to Fortune 500s. Learning Terraform is one of the most valuable skills a cloud engineer or DevOps professional can have in 2024.

1.3 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable configuration files, rather than through manual processes or interactive configuration tools.

Think of it this way: instead of logging into your cloud provider's website and clicking through menus to create a server, you write a text file that describes the server you want, and a tool (like Terraform) reads that file and creates it for you.

Traditional Infrastructure	Infrastructure as Code
Manual clicks in a web console	Declarative config files (.tf)
Hard to reproduce exactly	100% reproducible every time
No version history	Stored in Git with full history
Slow, error-prone	Fast, automated, reliable
Single person's knowledge	Team collaboration via code reviews
Hard to audit changes	Every change is tracked and reviewable

1.4 Declarative vs Imperative

These two terms describe two fundamentally different ways of telling a computer what to do:

Declarative (Terraform's approach)	Imperative (Shell scripts)
Tell WHAT you want	Tell HOW to do it step by step
'I want 3 servers running Ubuntu'	'Create server 1, then create server 2...'
Terraform figures out the steps	You write every step yourself
Easier to read and maintain	More complex and verbose
Idempotent — safe to run multiple times	Running twice may create duplicates

ANALOGY: GPS vs Directions

Declarative is like using GPS — you tell it where you want to go, and it figures out the route. Imperative is like giving someone turn-by-turn directions — 'Turn left, go 2 miles, turn right...' Terraform uses the GPS approach!

1.5 Terraform Use Cases

Terraform is incredibly versatile. Here are some of the most common real-world use cases:

- **Cloud Provisioning:** Create virtual machines, databases, networks, and load balancers on AWS, Azure, or GCP.
- **Multi-Cloud Management:** Manage resources across multiple cloud providers from a single configuration.
- **Kubernetes Cluster Setup:** Provision and configure Kubernetes clusters with all necessary components.
- **Network Automation:** Set up VPCs, subnets, routing tables, security groups, and firewalls automatically.
- **Security & Compliance:** Enforce security policies through code, ensuring consistent configurations.
- **Disaster Recovery:** Quickly spin up infrastructure in a different region if the primary region fails.
- **Development Environments:** Create short-lived sandbox environments for testing and development.

CH 2

Terraform Architecture

How Terraform works under the hood

Chapter 2: Terraform Architecture

Understanding Terraform's architecture helps you use it more effectively. This chapter walks through all the key components and how they work together to turn your configuration files into real infrastructure.

2.1 Terraform CLI

The Terraform CLI (Command Line Interface) is the tool you interact with directly. When you type commands like `terraform plan` or `terraform apply` in your terminal, you're using the Terraform CLI.

- It reads your `.tf` configuration files
- It communicates with Terraform Core to process your configs
- It displays output and status information to you
- It manages the connection to your configured providers

2.2 Terraform Core

Terraform Core is the brain of the system. It's the engine that runs behind the CLI and handles all the heavy lifting:

- Reads and parses your configuration files
- Builds a dependency graph to determine the correct order of operations
- Compares desired state (your config) with actual state (the state file)
- Plans what changes need to be made and in what order
- Coordinates with providers to make the actual changes

2.3 Providers

Providers are plugins that let Terraform communicate with different cloud platforms and services. Each provider contains the logic needed to interact with a specific platform's API.

- AWS Provider: Manages Amazon Web Services resources
- Azure Provider: Manages Microsoft Azure resources
- GCP Provider: Manages Google Cloud Platform resources
- Kubernetes Provider: Manages Kubernetes clusters
- GitHub Provider: Manages GitHub repositories and settings

There are over 3,000 providers available — covering virtually every cloud service and technology.

2.4 Resources

Resources are the actual infrastructure components you want to create and manage. They are the building blocks of your Terraform configuration.

- An AWS EC2 instance (a virtual machine)
- An S3 bucket (object storage)
- An Azure Virtual Network
- A Google Cloud SQL database
- A DNS record

2.5 State File

The Terraform state file (terraform.tfstate) is one of the most important concepts to understand. It's a JSON file that Terraform uses to keep track of what infrastructure it has created and manages.

ANALOGY: The Inventory List

Think of the state file as a warehouse inventory list. Every time you receive or ship goods (create or destroy resources), the inventory list is updated. Terraform checks this list before making changes to know what already exists.

DIAGRAM: Terraform Architecture Flow

Visualize this as a pipeline: USER types commands into the TERRAFORM CLI, which passes them to TERRAFORM CORE (the brain). The Core reads your CONFIG FILES (.tf), consults the STATE FILE to understand current infrastructure, then communicates with PROVIDERS (AWS, Azure, GCP plugins). Each Provider calls the respective CLOUD APIs to create/modify/destroy RESOURCES (servers, databases, networks). Arrows flow: User -> CLI -> Core -> Providers -> Cloud APIs -> Resources. State file sits between Core and Cloud, being updated after each operation.

CH 3

Installation & Setup

Getting Terraform running on your machine

Chapter 3: Installation & Setup

Let's get Terraform installed on your machine! This chapter provides step-by-step instructions for Windows, macOS, and Linux, plus important tips for managing Terraform versions.

3.1 Installing on Windows

1. Download the Terraform ZIP file from terraform.io/downloads
2. Extract the ZIP file to a folder (e.g., C:\terraform)
3. Add the folder to your System PATH environment variable
4. Open Command Prompt and run: `terraform --version`

```
# Windows (using Chocolatey package manager)
choco install terraform

# Verify installation
terraform --version
# Expected output: Terraform v1.x.x
```

3.2 Installing on macOS

5. Install Homebrew if not already installed: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
6. Run the Homebrew install command
7. Verify the installation

```
# macOS using Homebrew
brew tap hashicorp/tap
brew install hashicorp/tap/terraform

# Verify installation
terraform --version
```

3.3 Installing on Linux

```
# Ubuntu/Debian
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | \
  sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform

# Verify
terraform --version
```

3.4 Key Setup Commands

Command	Purpose
terraform init	Initialize working directory, download providers
terraform fmt	Format code to canonical style (auto-indentation)
terraform validate	Check configuration syntax for errors
terraform --version	Show installed Terraform version

TIP: Use Version Management

Use tools like tfenv (Terraform Version Manager) to switch between Terraform versions easily. Different projects may require different versions, and tfenv makes this seamless. Install it with: `git clone https://github.com/tfutils/tfenv.git ~/.tfenv`

CH 4

Providers

Connecting Terraform to cloud platforms

Chapter 4: Providers

Providers are the connectors between Terraform and the outside world. Without providers, Terraform wouldn't know how to talk to AWS, Azure, Google Cloud, or any other service. This chapter covers everything you need to know about providers.

4.1 What is a Provider?

A provider is a plugin that Terraform uses to interact with an API. It translates your Terraform resource declarations into actual API calls that create, update, or delete resources in your target platform.

ANALOGY: Language Translators

Imagine you speak only English and need to have conversations in French, Spanish, and Japanese simultaneously. Providers are your translators — each one speaks the 'language' of a specific cloud platform, letting Terraform communicate with all of them.

4.2 Configuring Providers

AWS Provider Example

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Create an EC2 instance using this provider
resource "aws_instance" "web_server" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"
}
```

Azure Provider Example

```
provider "azurerm" {
  features {}
  subscription_id = "your-subscription-id"
}

resource "azurerm_resource_group" "main" {
  name     = "my-resource-group"
  location = "East US"
}
```

GCP Provider Example

```
provider "google" {
  project = "my-project-id"
  region  = "us-central1"
  zone    = "us-central1-c"
}

resource "google_compute_instance" "vm_instance" {
  name         = "my-vm"
  machine_type = "e2-medium"
}
```

4.3 Multiple Providers

You can use multiple providers in a single Terraform project — great for multi-cloud setups:

```
# Using both AWS and Azure in one project
provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
}

# AWS resource
resource "aws_s3_bucket" "bucket" { ... }

# Azure resource
resource "azurerm_resource_group" "rg" { ... }
```

4.4 Provider Versioning

Always pin your provider versions to avoid unexpected breaking changes when providers release updates:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"    # Allow 5.x but not 6.x
    }
  }
  required_version = ">= 1.5.0" # Minimum Terraform version
}
```

TIP: Version Constraints

Use ~> for minor version flexibility (e.g., ~> 5.0 allows 5.x). Use = for exact versions when stability is critical. Always commit your .terraform.lock.hcl file to version control!

DIAGRAM: Provider Architecture

Draw a central TERRAFORM box in the middle. From it, draw arrows pointing outward to: AWS PROVIDER box (connecting to EC2, S3, RDS icons), AZURE PROVIDER box (connecting to VM, Blob Storage, SQL icons), GCP PROVIDER box (connecting to Compute Engine, Cloud Storage icons), and KUBERNETES PROVIDER box. Each Provider box should have a small 'Plugin' label. This shows how Terraform acts as a hub, with providers as spokes connecting to different cloud platforms.

CH 5

Resources

The building blocks of infrastructure

Chapter 5: Resources

Resources are the heart of Terraform. Every server, database, network, and storage bucket you create with Terraform is defined as a resource. This chapter teaches you how to write and manage resources effectively.

5.1 What is a Resource?

A resource represents a single piece of infrastructure — an AWS EC2 instance, an Azure SQL database, a DNS record, a Kubernetes namespace, or any other cloud component. Resources are declared in your .tf files using a specific syntax.

5.2 Resource Block Syntax

```
# Syntax:
# resource "<provider>_<type>" "<local_name>" { ... }

resource "aws_instance" "web_server" {
  ami           = "ami-0c02fb55956c7d316" # Amazon Machine Image
  instance_type = "t2.micro"               # Hardware type

  tags = {
    Name        = "My Web Server"
    Environment = "Production"
    Team        = "DevOps"
  }
}

# Accessing resource attributes after creation:
# aws_instance.web_server.public_ip -> The public IP of this server
# aws_instance.web_server.id        -> The instance ID
```

5.3 Resource Dependencies

Implicit Dependencies (Recommended)

When one resource references another, Terraform automatically knows it must create the referenced resource first:

```
resource "aws_vpc" "main" {
```

```
    cidr_block = "10.0.0.0/16"
  }

  resource "aws_subnet" "public" {
    vpc_id      = aws_vpc.main.id # Implicit dependency!
    cidr_block = "10.0.1.0/24"
    # Terraform knows: create aws_vpc.main BEFORE aws_subnet.public
  }
```

Explicit Dependencies (depends_on)

Use `depends_on` when a dependency exists but isn't expressed through resource references:

```
resource "aws_iam_role_policy" "example" {
  name = "example_policy"
  role = aws_iam_role.example.id
  policy = "...
}

resource "aws_instance" "example" {
  ami          = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"

  depends_on = [aws_iam_role_policy.example]
  # Explicitly telling Terraform: wait for the IAM policy first
}
```

TIP: Prefer Implicit Dependencies

Wherever possible, use implicit dependencies (referencing resource attributes) over explicit `depends_on`. Implicit dependencies are self-documenting and easier to maintain. Use `depends_on` only when there's no way to express the dependency through references.

CH 6

Variables

Making your configs flexible and reusable

Chapter 6: Variables

Hard-coding values into your Terraform configurations is a recipe for headaches. Variables make your configurations flexible, reusable, and easy to maintain. This chapter covers everything about Terraform variables.

6.1 Input Variables

Input variables let you parameterize your Terraform modules. Instead of hard-coding values, you define variables that can be set at runtime or through variable files.

```
# Defining variables in variables.tf
variable "region" {
  type        = string
  description = "AWS region to deploy resources"
  default     = "us-east-1"
}

variable "instance_count" {
  type        = number
  description = "Number of EC2 instances to create"
  default     = 2
}

variable "enable_monitoring" {
  type        = bool
  description = "Enable detailed monitoring"
  default     = true
}

variable "allowed_ports" {
  type        = list(number)
  description = "List of ports to allow in security group"
  default     = [80, 443, 8080]
}

variable "tags" {
  type        = map(string)
  description = "Tags to apply to all resources"
  default     = {
    Team    = "DevOps"
    Project = "MyApp"
  }
}
```

6.2 Variable Types

Type	Example
string	"us-east-1", "production", "t2.micro"
number	3, 80, 443, 1024
bool	true, false
list(string)	["subnet-1", "subnet-2", "subnet-3"]
map(string)	{ env = "prod", team = "devops" }
object({...})	Complex structured types with named attributes
any	Accept any type (use sparingly)

6.3 Using .tfvars Files

```
# terraform.tfvars (auto-loaded by Terraform)
region          = "eu-west-1"
instance_count  = 5
enable_monitoring = false

# production.tfvars (specify with -var-file flag)
region          = "us-east-1"
instance_count  = 10

# Apply with specific vars file:
# terraform apply -var-file="production.tfvars"
```

6.4 Sensitive Variables

Never store secrets like passwords or API keys in plain text! Use sensitive = true to prevent values from appearing in logs and outputs:

```
variable "db_password" {
  type          = string
  description    = "Database password"
  sensitive      = true # Won't show in terraform plan/apply output
}

# Set via environment variable (most secure):
# export TF_VAR_db_password="my-secret-password"
# terraform apply
```

SECURITY WARNING

Never commit .tfvars files containing real secrets to version control. Use environment variables (TF_VAR_*) or secret management tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault for sensitive values.

CH 7

Outputs

Exposing values from your infrastructure

Chapter 7: Outputs

After Terraform creates your infrastructure, you often need to know certain values — like the IP address of a newly created server, or the URL of a load balancer. Outputs let you extract and display these values.

7.1 Defining Outputs

```
# outputs.tf
output "web_server_ip" {
  value      = aws_instance.web_server.public_ip
  description = "The public IP address of the web server"
}

output "load_balancer_url" {
  value      = "http://${aws_lb.main.dns_name}"
  description = "URL to access the application"
}

output "database_endpoint" {
  value      = aws_db_instance.main.endpoint
  description = "Database connection endpoint"
  sensitive  = true # Won't display in terminal output
}
```

7.2 Viewing Outputs

```
# After terraform apply, outputs are shown automatically
# You can also view them anytime:
terraform output

# View a specific output:
terraform output web_server_ip

# Get output in JSON format (useful in scripts):
terraform output -json
```

7.3 Outputs in Module Communication

Outputs are essential when using modules. A child module exposes values through outputs, which the parent module can then use as inputs to other modules:

```
# In child module (modules/networking/outputs.tf):
output "vpc_id" {
  value = aws_vpc.main.id
}

# In root module (main.tf):
module "networking" {
  source = "./modules/networking"
}

module "compute" {
  source = "./modules/compute"
  vpc_id = module.networking.vpc_id # Using child module output!
}
```

CH 8

State Management

How Terraform tracks your infrastructure

Chapter 8: State Management

The Terraform state file is one of the most critical concepts to master. Without understanding state, you can accidentally destroy resources, cause conflicts with teammates, or lose track of what Terraform manages. This chapter covers everything.

8.1 What is Terraform State?

The state file (terraform.tfstate) is a JSON file that maps your Terraform resources to real-world infrastructure. Every time Terraform runs, it reads this file to understand the current state of your managed resources.

ANALOGY: The Blueprint Archive

Imagine a construction company that keeps a detailed file for every building they've ever built — blueprints, current state, modifications, and ownership. The state file is exactly that — Terraform's master record of everything it has built and manages.

8.2 Local vs Remote State

Local State	Remote State
Stored on your local machine	Stored in shared location (S3, GCS, Azure Blob)
Good for solo projects	Required for team collaboration
No locking — can cause conflicts	State locking prevents conflicts
Risk of accidental deletion	Versioned and backed up automatically
Default behavior	Must be configured explicitly

8.3 State Locking

When multiple team members run Terraform simultaneously, they could corrupt the state file. State locking prevents this by allowing only one Terraform operation at a time.

```
# State locking is automatic with DynamoDB backend (AWS):
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
```

```
key          = "prod/terraform.tfstate"
region       = "us-east-1"
dynamodb_table = "terraform-locks" # Enables state locking
encrypt      = true
}
}
```

8.4 Security Best Practices for State

- Enable encryption at rest for remote state storage
- Restrict access using IAM policies — only Terraform runners should have access
- Never store state files in public Git repositories
- Enable versioning on your state storage bucket for rollback capability
- Use separate state files for different environments (dev, staging, prod)

DIAGRAM: State Lifecycle

Draw a cycle with these stages: 1) TERRAFORM APPLY runs, 2) Core reads current STATE FILE, 3) Core calls PROVIDER API to see real resources, 4) Core compares desired config vs actual state, 5) Makes required changes via API, 6) UPDATES STATE FILE with new resource details. Show this as a circular flow with arrows between each step. Add a separate box showing REMOTE STATE STORAGE options: S3 Bucket, Azure Blob, GCS, Terraform Cloud — connected to the main cycle via a 'sync' arrow.

CH 9

Backends

Configuring state storage and remote operations

Chapter 9: Backends

Backends define where Terraform stores its state and, optionally, where Terraform operations are executed. Choosing the right backend is crucial for team collaboration and security.

9.1 Local Backend (Default)

```
# Explicit local backend configuration
terraform {
  backend "local" {
    path = "terraform.tfstate"
  }
}
```

9.2 S3 Backend (AWS)

```
terraform {
  backend "s3" {
    bucket      = "my-company-terraform-state"
    key         = "environments/production/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "terraform-state-lock" # For locking
    encrypt     = true

    # Optional: Use IAM role instead of access keys
    role_arn = "arn:aws:iam::123456789:role/TerraformRole"
  }
}
```

9.3 Terraform Cloud Backend

```
terraform {
  cloud {
    organization = "my-company"
    workspaces {
      name = "production"
    }
  }
}
```

```
}
```

BEST PRACTICE: Always Use Remote Backends for Teams

If you're working with any team — even just one other person — always use a remote backend. Local backends cause state file conflicts that are difficult and sometimes impossible to resolve. Terraform Cloud offers a free tier for small teams!

CH 10**Core Commands**

The Terraform workflow commands

Chapter 10: Core Terraform Commands

The Terraform workflow revolves around a small set of commands that you'll use every day. Mastering these commands is essential for effective Terraform use.

10.1 terraform init

The first command you run in any Terraform project. It initializes the working directory, downloads required providers, and sets up the backend.

```
# Basic initialization
terraform init

# Upgrade providers to latest allowed versions
terraform init -upgrade

# Reconfigure backend
terraform init -reconfigure

# Output after init includes:
# - Provider download confirmation
# - Backend configuration
# - Ready message
```

ANALOGY: Setting Up Your Kitchen

terraform init is like setting up your kitchen before cooking. You stock the pantry (download providers), check your equipment (validate backend), and make sure everything is in place before you start cooking (creating infrastructure).

10.2 terraform plan

Shows what Terraform **WOULD** do without actually doing it. Always run plan before apply to review changes.

```
# Standard plan
terraform plan

# Save plan to file (recommended for production)
terraform plan -out=tfplan
```

```
# Plan with variable override
terraform plan -var="instance_type=t3.medium"

# Plan output shows:
# + resource to be created (green)
# - resource to be destroyed (red)
# ~ resource to be modified (yellow)
```

10.3 terraform apply

```
# Apply with interactive approval prompt
terraform apply

# Apply a saved plan (no prompt needed)
terraform apply tfplan

# Auto-approve (use carefully! Good for CI/CD)
terraform apply -auto-approve

# Apply with variable override
terraform apply -var="environment=production"
```

10.4 terraform destroy

```
# Destroy all managed resources (prompts for confirmation)
terraform destroy

# Destroy specific resource only
terraform destroy -target=aws_instance.web_server

# Auto-approve destroy (dangerous! use with extreme caution)
terraform destroy -auto-approve
```

DANGER: terraform destroy

The destroy command permanently deletes all resources managed by Terraform. Always review the plan carefully before confirming. In production environments, consider using lifecycle { prevent_destroy = true } to add an extra safety net.

10.5 terraform show & terraform state

```
# Show current state in human-readable format
terraform show

# Show a saved plan
terraform show tfplan
```

```
# List all resources in state
terraform state list

# Show details of specific resource
terraform state show aws_instance.web_server

# Remove resource from state (without destroying it)
terraform state rm aws_instance.old_server

# Move resource in state (rename)
terraform state mv aws_instance.old aws_instance.new
```

CH 11

Modules

Organizing and reusing Terraform code

Chapter 11: Modules

Modules are the key to writing scalable, maintainable Terraform code. Just like functions in programming, modules let you package and reuse configurations. This chapter covers everything from basic module concepts to the Terraform Module Registry.

11.1 What is a Module?

A module is a container for multiple resources that are used together. Every Terraform configuration has at least one module — the root module (the files in your working directory). Additional modules are called child modules.

ANALOGY: LEGO Blocks

Modules are like LEGO blocks — pre-designed pieces that you can combine in different ways to build larger structures. Instead of designing every brick from scratch, you use pre-built blocks and connect them together. You can even share your blocks (modules) with others!

11.2 Module Structure

```
# Typical module directory structure:
modules/
  networking/
    main.tf      # Resources
    variables.tf # Input variables
    outputs.tf   # Output values
  compute/
    main.tf
    variables.tf
    outputs.tf
  database/
    main.tf
    variables.tf
    outputs.tf
main.tf          # Root module - calls child modules
```

11.3 Using Modules

```
# main.tf (root module)
```

```
module "networking" {
  source = "../modules/networking" # Local module

  # Pass inputs to the module
  vpc_cidr = "10.0.0.0/16"
  region   = var.region
}

module "compute" {
  source = "../modules/compute"

  vpc_id      = module.networking.vpc_id # Use output from networking module
  subnet_id   = module.networking.subnet_id

  instance_type = "t3.medium"
  instance_count = 3
}

# Use a public module from Terraform Registry:
module "eks" {
  source = "terraform-aws-modules/eks/aws" # Registry module
  version = "~> 19.0"

  cluster_name = "my-eks-cluster"
  vpc_id        = module.networking.vpc_id
}
```

11.4 The Terraform Module Registry

The Terraform Registry (registry.terraform.io) hosts thousands of pre-built, community-maintained modules. Before writing a module from scratch, always check if one already exists!

- AWS modules: VPC, EKS, RDS, EC2, IAM, and hundreds more
- Azure modules: VNet, AKS, SQL Database, and more
- GCP modules: GKE, Cloud SQL, VPC, and more
- All registry modules are versioned and have documentation

CH 12

Meta-Arguments

Advanced resource configuration options

Chapter 12: Resource Meta-Arguments

Meta-arguments are special arguments available on every resource type. They control how Terraform handles resources, rather than configuring the resource itself. This chapter covers the five key meta-arguments.

12.1 depends_on

Creates explicit dependencies between resources when Terraform can't automatically determine them:

```
resource "aws_instance" "app" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"

  depends_on = [
    aws_iam_role_policy.app_policy,
    aws_security_group.app_sg
  ]
}
```

12.2 count

Creates multiple instances of a resource from a single block:

```
# Create 5 identical EC2 instances
resource "aws_instance" "servers" {
  count      = 5
  ami       = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"

  tags = {
    Name = "server-${count.index + 1}" # server-1, server-2, etc.
  }
}

# Access specific instance: aws_instance.servers[0]
# Access all instances:    aws_instance.servers[*].id
```

12.3 for_each

Creates resources for each item in a set or map — more flexible than count:

```
variable "environments" {
  default = {
    dev  = "t2.micro"
    staging = "t2.small"
    prod = "t3.medium"
  }
}

resource "aws_instance" "env_servers" {
  for_each      = var.environments
  ami           = "ami-0c02fb55956c7d316"
  instance_type = each.value # t2.micro, t2.small, or t3.medium

  tags = {
    Name           = "server-${each.key}" # server-dev, server-staging, server-prod
    Environment    = each.key
  }
}
```

count vs for_each

Use count when you want N identical resources. Use for_each when each resource has different properties (like different names or sizes). for_each is generally preferred because removing an item in the middle doesn't cause all subsequent resources to be recreated.

12.4 lifecycle

```
resource "aws_instance" "critical_server" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t3.large"

  lifecycle {
    # Create replacement BEFORE destroying old one
    create_before_destroy = true

    # Prevent accidental destruction of this resource
    prevent_destroy = true

    # Don't update these attributes even if config changes
    ignore_changes = [
      ami,
      tags["LastModified"]
    ]
  }
}
```

CH 13

Data Sources

Reading existing infrastructure

Chapter 13: Data Sources

Data sources let you read information from existing infrastructure — resources that exist outside of your Terraform configuration. This is extremely useful for referencing pre-existing resources.

13.1 Resources vs Data Sources

Resources (resource {})	Data Sources (data {})
Creates, updates, destroys infrastructure	Only reads existing infrastructure
Terraform manages the lifecycle	Terraform doesn't manage it — just reads it
Changes tracked in state	Not tracked in state
Use to build new things	Use to reference existing things

13.2 Common Data Source Examples

```
# Fetch the latest Amazon Linux 2 AMI automatically
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

# Use it in a resource:
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id # Always up-to-date!
  instance_type = "t2.micro"
}

# Fetch an existing VPC by name tag
data "aws_vpc" "existing" {
  filter {
    name   = "tag:Name"
    values = ["my-existing-vpc"]
  }
}
```

```
}

# Fetch current AWS account info
data "aws_caller_identity" "current" {}
# Usage: data.aws_caller_identity.current.account_id
```

TIP: Avoid Hardcoding IDs

Instead of hardcoding resource IDs like AMI IDs or VPC IDs (which can change!), use data sources to look them up dynamically. This makes your code much more portable and maintainable.

CH 14

Expressions & Functions

Dynamic configuration with HCL

Chapter 14: Expressions & Functions

HCL (HashiCorp Configuration Language) includes powerful expressions and built-in functions that let you write dynamic, DRY (Don't Repeat Yourself) configurations. This chapter covers the most useful ones.

14.1 Interpolation Syntax

```
# Reference a resource attribute
ami = aws_instance.web.ami

# String interpolation with ${ }
name = "server-${var.environment}-${count.index}"

# Complex expression
bucket_name = "${var.project}-${var.environment}-${data.aws_caller_identity.current.account_id}"
```

14.2 Built-in Functions

Function	Example & Result
length()	length(["a","b","c"]) → 3
join()	join("-", ["a","b","c"]) → "a-b-c"
split()	split(",", "a,b,c") → ["a","b","c"]
lookup()	lookup(map,"key","default") → value
element()	element(["a","b","c"], 1) → "b"
concat()	concat([1,2],[3,4]) → [1,2,3,4]
flatten()	flatten([[1,2],[3,4]]) → [1,2,3,4]
toset()	toset(["a","b","a"]) → {"a","b"}
upper()/lower()	upper("hello") → "HELLO"
format()	format("%s-%d", "server", 1) → "server-1"
cidrsubnet()	cidrsubnet("10.0.0.0/16",8,1) → "10.0.1.0/24"
tostring()	tostring(42) → "42"

14.3 Conditional Expressions

```
# Syntax: condition ? true_value : false_value

# Use t3.large in production, t2.micro otherwise
instance_type = var.environment == "production" ? "t3.large" : "t2.micro"

# Enable deletion protection only in prod
deletion_protection = var.environment == "production" ? true : false

# Choose AMI based on region
ami = var.region == "us-east-1" ? "ami-abc123" : "ami-def456"

# Conditional resource count (create 0 or 1)
resource "aws_instance" "monitoring" {
  count = var.enable_monitoring ? 1 : 0
  # This resource only exists if enable_monitoring is true
  ...
}
```

14.4 For Expressions

```
# Transform a list
variable "instance_names" {
  default = ["web", "api", "db"]
}

locals {
  uppercase_names = [for name in var.instance_names : upper(name)]
  # Result: ["WEB", "API", "DB"]

  # Filter: only names longer than 2 chars
  long_names = [for name in var.instance_names : name if length(name) > 2]
  # Result: ["web", "api"]

  # Create a map
  name_map = { for name in var.instance_names : name => "${name}-server" }
  # Result: { web = "web-server", api = "api-server", db = "db-server" }
}
```

CH 15

Workspaces

Managing multiple environments

Chapter 15: Workspaces

Workspaces allow you to manage multiple environments (dev, staging, production) using the same Terraform configuration with isolated state files for each.

15.1 Workspace Concepts

ANALOGY: Parallel Universes

Think of workspaces like parallel universes of your infrastructure. Each workspace runs the same Terraform code but has its own independent state, so changes in 'dev' don't affect 'production' — they're completely isolated.

15.2 Workspace Commands

```
# List all workspaces (* indicates current)
terraform workspace list

# Output:
# * default
#   development
#   staging
#   production

# Create a new workspace
terraform workspace new development

# Switch to a workspace
terraform workspace select production

# Show current workspace
terraform workspace show

# Delete a workspace
terraform workspace delete development
```

15.3 Using Workspace in Configuration

```
# Use workspace name in your config
```

```
resource "aws_instance" "server" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = terraform.workspace == "production" ? "t3.large" : "t2.micro"

  tags = {
    Environment = terraform.workspace
    Name        = "server-${terraform.workspace}"
  }
}

locals {
  # Different settings per workspace
  config = {
    dev      = { instance_type = "t2.micro", count = 1 }
    staging  = { instance_type = "t2.small", count = 2 }
    production = { instance_type = "t3.medium", count = 5 }
  }

  current = local.config[terraform.workspace]
}
```

CH 16

Terraform Import

Bringing existing infra under Terraform control

Chapter 16: Terraform Import

What if you have existing infrastructure that wasn't created by Terraform? The terraform import command lets you bring those resources under Terraform management without recreating them.

16.1 How Import Works

Importing does NOT generate configuration files automatically (in older Terraform versions). You must first write the resource block in your .tf file, then import the actual resource into the state.

```
# Step 1: Write the resource block in main.tf
resource "aws_instance" "existing_server" {
  # You'll fill in the details after importing
}

# Step 2: Import the real resource into Terraform state
# Syntax: terraform import <resource_type>.<name> <real_resource_id>
terraform import aws_instance.existing_server i-1234567890abcdef0

# Step 3: Run terraform show to see actual resource config
terraform show

# Step 4: Update your .tf file to match the imported state
# Step 5: Run terraform plan to verify no changes are needed
```

16.2 Terraform 1.5+ Generated Config

```
# New in Terraform 1.5: import blocks with generated config
import {
  to = aws_instance.existing_server
  id = "i-1234567890abcdef0"
}

# Run with -generate-config-out to auto-generate the resource block:
terraform plan -generate-config-out=generated.tf
```

16.3 Import Limitations & Tips

- Not all resources support import — check provider documentation

- Import only affects state — you must still write matching configuration
- Some resource attributes cannot be read after creation (e.g., passwords)
- Test imports in a non-production environment first
- Run terraform plan after import to verify no unintended changes

CH 17**Drift Detection**

Keeping infrastructure in sync

Chapter 17: Drift Detection

Drift occurs when your real infrastructure no longer matches what Terraform thinks it should look like. This can happen when someone makes manual changes directly in the cloud console.

17.1 What is Drift?

ANALOGY: Ship Off Course

Imagine setting your ship on autopilot to sail to a destination. If someone moves the ship slightly while autopilot is off, you've drifted off course. Terraform drift is the same — someone made manual changes that now differ from your intended configuration.

17.2 Detecting Drift

```
# terraform plan automatically detects drift!
# It compares:
# 1. Your .tf configuration (desired state)
# 2. The state file (what Terraform thinks exists)
# 3. Real infrastructure (what actually exists)

terraform plan
# Terraform will flag any drift as changes to be made

# Force refresh of state from real infrastructure:
terraform apply -refresh-only
# This updates state file WITHOUT making changes

# Approve the refresh to update state:
terraform apply -refresh-only -auto-approve
```

17.3 Preventing Drift

- Restrict direct console access — use Terraform for ALL changes
- Use IAM policies to limit who can modify resources directly
- Run terraform plan in CI/CD pipelines regularly to catch drift early
- Set up automated drift detection with Terraform Cloud or custom scripts
- Document change management processes to prevent unauthorized modifications

CH 18

Terraform Cloud & Enterprise

Team collaboration at scale

Chapter 18: Terraform Cloud & Enterprise

Terraform Cloud is HashiCorp's managed service that adds team collaboration features, remote execution, policy enforcement, and more. Enterprise adds additional features for large organizations.

18.1 Key Features

Feature	Description
Remote Runs	Terraform plans and applies run in a consistent, isolated environment in the cloud
Remote State	State is stored securely in the cloud with locking, versioning, and access controls
Sentinel Policies	Policy-as-code framework to enforce compliance and governance rules
Team Collaboration	Role-based access control (RBAC) for managing team permissions
VCS Integration	Connect to GitHub/GitLab/Bitbucket for automatic plan on pull requests
Cost Estimation	Estimate the monthly cost of infrastructure changes before applying
Audit Logs	Complete audit trail of who ran what and when
Private Registry	Host your own private module registry for your organization

18.2 Sentinel Policies

Sentinel is HashiCorp's policy-as-code framework that runs checks after terraform plan but before terraform apply. It can enforce rules like 'all resources must have cost tags' or 'EC2 instances cannot be larger than t3.xlarge'.

```
# Example Sentinel policy: require specific tags
import "tfplan/v2" as tfplan

# Require all resources to have an "Environment" tag
main = rule {
```

```
all tfplan.resource_changes as _, rc {  
    rc.change.after.tags["Environment"] is not null  
}  
}
```

CH 19**Security**

Protecting your infrastructure and secrets

Chapter 19: Security Best Practices

Security is not an afterthought in Terraform — it must be built into your workflow from the start. This chapter covers the most important security practices for Terraform users.

19.1 Protecting State Files

- Never store state files in public repositories — they contain sensitive resource details
- Enable encryption at rest and in transit for remote state storage
- Use IAM roles and policies to restrict who can access state files
- Enable versioning on S3 buckets storing state for rollback capability
- Rotate access keys regularly and use short-lived credentials where possible

19.2 Handling Sensitive Variables

```
# Mark variables as sensitive
variable "api_key" {
  type      = string
  sensitive = true
}

# Use environment variables (most secure approach)
# export TF_VAR_api_key="your-secret-key"

# Use AWS Secrets Manager via data source
data "aws_secretsmanager_secret_version" "api_key" {
  secret_id = "prod/myapp/api-key"
}

# Reference without exposing in code:
locals {
  api_key =
  jsondecode(data.aws_secretsmanager_secret_version.api_key.secret_string)["key"]
}
```

19.3 IAM Least Privilege

The principle of least privilege means your Terraform execution role should only have permissions for exactly what it needs — nothing more.

- Create dedicated IAM roles for Terraform with only required permissions
- Use separate roles for different environments (dev vs prod)
- Regularly audit and review IAM permissions
- Use IAM conditions to restrict access by IP address or time
- Prefer IAM roles over long-term access keys

SECURITY TIP: Scan Your Code

Use tools like tfsec, checkov, or terrascan to automatically scan your Terraform code for security misconfigurations before applying. These tools can catch issues like public S3 buckets, unencrypted databases, and overly permissive security groups.

CH 20

Testing & Validation

Ensuring your code is correct

Chapter 20: Testing & Validation

Testing infrastructure code is just as important as testing application code. This chapter covers built-in validation commands and third-party testing tools.

20.1 Built-in Validation Commands

Command	What it Checks
terraform validate	Syntax errors, invalid resource types, missing required fields
terraform fmt	Code formatting — ensures consistent style (4-space indent, aligned = signs)
terraform plan	Logic errors, dependency issues, and shows what would change
terraform apply -refresh-only	Drift between state and real infrastructure

20.2 Variable Validation

```
variable "environment" {
  type = string

  validation {
    condition     = contains(["dev", "staging", "production"], var.environment)
    error_message = "Environment must be dev, staging, or production."
  }
}

variable "instance_count" {
  type = number

  validation {
    condition     = var.instance_count >= 1 && var.instance_count <= 20
    error_message = "Instance count must be between 1 and 20."
  }
}
```

20.3 Testing Tools

- **terraform-compliance** terraform-compliance: BDD-style testing for infrastructure code — define policies as plain English
- **Terratest** terratest: Go-based framework for writing automated tests that actually deploy and test real infrastructure
- **tfsec** tfsec: Static analysis tool that scans for security misconfigurations
- **Checkov** checkov: Policy-as-code scanner for Terraform and other IaC tools
- **Infracost** infracost: Estimate cloud costs before applying changes

CH 21

Best Practices

Writing production-quality Terraform code

Chapter 21: Best Practices

These best practices are distilled from real-world experience with Terraform at scale. Follow these guidelines to write code that's maintainable, safe, and professional.

21.1 Code Organization

```
# Recommended file structure for a Terraform project:
project/
  main.tf           # Core resources
  variables.tf      # All input variable declarations
  outputs.tf        # All output declarations
  providers.tf      # Provider configuration
  versions.tf       # Required providers and versions
  locals.tf         # Local value computations

  modules/
    networking/     # VPC, subnets, routing
    compute/        # EC2, ASG, ECS
    database/       # RDS, DynamoDB
    security/       # IAM, security groups

  environments/
    dev/            # Dev-specific var files
    staging/
    production/
```

21.2 The Golden Rules of Terraform

8. **Remote Backends:** Always use remote backends for team projects.
9. **Version Pinning:** Pin provider versions — never leave them unconstrained.
10. **Plan Before Apply:** Always run terraform plan before terraform apply.
11. **Modular Design:** Use modules to encapsulate repeated patterns.
12. **No Hardcoded Secrets:** Never hardcode secrets — use environment variables or secret managers.
13. **Naming Conventions:** Use consistent naming conventions across all resources.
14. **State Isolation:** Separate state by environment to prevent cross-contamination.
15. **Version Control:** Store all Terraform code in version control (Git).
16. **Documentation:** Document your modules with README files and variable descriptions.

17. **Test First:** Test in dev before applying to production.

21.3 Naming Conventions

```
# Use snake_case for all names
resource "aws_instance" "web_server" {}      ✓
resource "aws_instance" "webServer" {}      X (camelCase)
resource "aws_instance" "web-server" {}     X (hyphens)

# Be descriptive
resource "aws_security_group" "web_server_sg" {}  ✓
resource "aws_security_group" "sg1" {}           X

# Include environment in resource names/tags
tags = {
  Name          = "web-server-${var.environment}"
  Environment    = var.environment
  Project        = var.project_name
  ManagedBy      = "Terraform"
}
```

CH 22

Terraform vs Other Tools

Understanding the IaC landscape

Chapter 22: Terraform vs Other Tools

Terraform is not the only Infrastructure as Code tool available. Here's a brief introduction to the main alternatives, helping you understand when each might be appropriate.

Tool	Key Characteristics
Terraform (HashiCorp)	Multi-cloud, declarative, large provider ecosystem, HCL language, open-source core
Ansible (Red Hat)	Primarily configuration management and app deployment, agentless, YAML playbooks, imperative
CloudFormation (AWS)	AWS-native only, JSON/YAML format, deep AWS integration, free with AWS account
ARM Templates (Azure)	Azure-native only, JSON format, deep Azure integration, free with Azure account
Pulumi	Multi-cloud like Terraform but uses real programming languages (Python, TypeScript, Go)
CDK (AWS)	AWS-native, uses real programming languages, generates CloudFormation underneath
Bicep (Azure)	Azure-native, cleaner syntax than ARM templates, compiles to ARM JSON

When to Use What

Use Terraform for multi-cloud or when you want a proven, widely-adopted tool with excellent community support. Use CloudFormation/ARM when you're fully committed to a single cloud and want the deepest possible integration. Use Ansible for configuring existing servers and deploying applications. Many teams use Terraform and Ansible together!

CH 23**Terraform in DevOps**

CI/CD integration and GitOps workflows

Chapter 23: Terraform in DevOps

Terraform is most powerful when integrated into automated DevOps workflows. This chapter shows how to use Terraform with CI/CD pipelines, Git workflows, and other DevOps tools.

23.1 GitOps Workflow

In a GitOps workflow, your Git repository is the single source of truth for your infrastructure:

18. Developer writes Terraform code in a feature branch
19. Opens a Pull Request — CI automatically runs terraform plan and posts results
20. Team reviews the plan changes in the PR
21. PR is merged to main — CI/CD runs terraform apply automatically
22. State file is updated and stored in remote backend
23. Repeat for next change

23.2 GitHub Actions CI/CD Example

```
# .github/workflows/terraform.yml
name: Terraform CI/CD

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v2
        with:
          terraform_version: 1.5.0

      - name: Terraform Init
        run: terraform init
```

```
- name: Terraform Fmt Check
  run: terraform fmt -check

- name: Terraform Validate
  run: terraform validate

- name: Terraform Plan
  run: terraform plan -no-color
  if: github.event_name == 'pull_request'

- name: Terraform Apply
  run: terraform apply -auto-approve
  if: github.ref == 'refs/heads/main'
```

23.3 Kubernetes Integration

Terraform can provision Kubernetes clusters and manage Kubernetes resources:

```
# Create an EKS cluster with Terraform
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 19.0"

  cluster_name = "my-cluster"
  vpc_id       = module.vpc.vpc_id
}

# Use Kubernetes provider to deploy workloads
provider "kubernetes" {
  host = module.eks.cluster_endpoint
  token = data.aws_eks_cluster_auth.main.token
}

resource "kubernetes_namespace" "app" {
  metadata {
    name = "my-app"
  }
}
```

CH 24

Troubleshooting

Solving common Terraform problems

Chapter 24: Troubleshooting

Even experienced Terraform users run into issues. This chapter covers the most common problems and how to diagnose and fix them.

24.1 Common Issues & Solutions

Init Failures

ANALOGY: Missing Ingredients

A terraform init failure is like trying to cook but the grocery store is closed — you're missing ingredients (providers). Check your internet connection, provider source URLs, and ensure your terraform { required_providers {} } block is correctly formatted.

```
# Common init failure causes:
# 1. Incorrect provider source
source = "hashicorp/aws"      ✓
source = "aws"                ✗

# 2. Network issues - check connectivity
curl -I https://registry.terraform.io

# 3. Enable debug logging for more detail:
TF_LOG=DEBUG terraform init
```

Provider Authentication Errors

Provider errors usually mean Terraform can't authenticate with your cloud provider:

```
# AWS - check credentials
aws sts get-caller-identity # Should return your account info

# Set credentials explicitly
export AWS_ACCESS_KEY_ID="your-key"
export AWS_SECRET_ACCESS_KEY="your-secret"
export AWS_DEFAULT_REGION="us-east-1"

# Enable verbose logging
TF_LOG=DEBUG terraform plan 2>&1 | grep -i "error\|credential"
```

State Lock Issues

ANALOGY: Door Locked by Someone Else

A state lock is like someone else holding the key to a shared room. You need to either wait for them to finish (the lock should release automatically) or forcibly remove the lock if you're sure the other process is no longer running.

```
# State lock error message looks like:
# Error: Error locking state: ConditionalCheckFailedException

# Force unlock (only if you're sure the locking process is gone!):
terraform force-unlock <LOCK_ID>

# Find Lock ID in error message or DynamoDB table
```

Dependency Conflicts

ANALOGY: Traffic at a Junction

Dependency conflicts are like two cars trying to enter the same lane at once. You need clear rules about who goes first. Use explicit `depends_on` or reorganize your resource references to create a clear ordering.

Common Error Messages

Error	Solution
Error: Unsupported argument	Check resource documentation for valid arguments
Error: Missing required argument	Add the required field to your resource block
Error: Invalid value for variable	Check validation rules or variable type constraints
Error: Resource already exists	Import the existing resource or use a different name
Error: Cycle detected	Remove circular dependencies between resources
Error: No valid credential sources	Configure cloud provider authentication credentials

CH 25**Real-World Use Cases**

Putting it all together with practical examples

Chapter 25: Real-World Use Cases

This final chapter brings together everything you've learned with comprehensive real-world examples. These are the kinds of problems you'll encounter in actual professional environments.

25.1 Provisioning a Complete Web Application Stack on AWS

```
# Complete 3-tier web app: Load Balancer + EC2 + RDS
# networking/main.tf
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "production-vpc" }
}

resource "aws_subnet" "public" {
  count          = 2
  vpc_id         = aws_vpc.main.id
  cidr_block     = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index)
  availability_zone = data.aws_availability_zones.available.names[count.index]
}

# Application Load Balancer
resource "aws_lb" "main" {
  name          = "production-alb"
  internal      = false
  load_balancer_type = "application"
  subnets     = aws_subnet.public[*].id
}

# Auto Scaling Group for web servers
resource "aws_autoscaling_group" "web" {
  min_size         = 2
  max_size         = 10
  desired_capacity = 3
  vpc_zone_identifier = aws_subnet.public[*].id
}

# RDS Database
resource "aws_db_instance" "main" {
  engine          = "postgres"
  engine_version = "15.0"
  instance_class = "db.t3.medium"
  allocated_storage = 100
}
```

```
storage_encrypted = true
deletion_protection = true
}
```

25.2 Multi-Cloud Deployment

```
# Deploy app to both AWS and Azure for high availability
provider "aws" { region = "us-east-1" }
provider "azurerm" { features {} }

# AWS side
resource "aws_instance" "app" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
}

# Azure side (as failover)
resource "azurerm_linux_virtual_machine" "app" {
  name                        = "app-vm-azure"
  resource_group_name        = azurerm_resource_group.main.name
  size                       = "Standard_D2s_v3"

  os_disk { caching = "ReadWrite"; storage_account_type = "Premium_LRS" }
  source_image_reference {
    publisher = "Canonical"; offer = "UbuntuServer"
    sku       = "20.04-LTS"; version = "latest"
  }
}
```

25.3 EKS Kubernetes Cluster Setup

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  name    = "my-eks-vpc"
  cidr    = "10.0.0.0/16"
  azs     = ["us-east-1a", "us-east-1b", "us-east-1c"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24", "10.0.13.0/24"]
  enable_nat_gateway = true
}

module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 19.0"
  cluster_name    = "production-cluster"
  cluster_version = "1.27"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
}
```

```
eks_managed_node_groups = {
  general = {
    desired_size = 3
    min_size     = 2
    max_size     = 10
    instance_types = ["t3.medium"]
  }
}
```

25.4 Network & Security Automation

```
# Automated security group rules from a variable
variable "allowed_services" {
  default = {
    http = { port = 80, protocol = "tcp" }
    https = { port = 443, protocol = "tcp" }
    ssh = { port = 22, protocol = "tcp" }
  }
}

resource "aws_security_group_rule" "allow_services" {
  for_each = var.allowed_services

  security_group_id = aws_security_group.web.id
  type              = "ingress"
  from_port         = each.value.port
  to_port           = each.value.port
  protocol          = each.value.protocol
  cidr_blocks       = ["0.0.0.0/0"]
  description       = "Allow ${each.key} traffic"
}
```

25.5 Complete Infrastructure Automation Workflow

DIAGRAM: End-to-End Terraform DevOps Workflow

Draw a horizontal pipeline with these stages connected by arrows: 1) DEVELOPER WRITES CODE (laptop icon, .tf files), 2) GIT PUSH (GitHub icon, feature branch), 3) PULL REQUEST OPENED (PR icon), 4) CI RUNS PLAN (GitHub Actions, shows 'terraform plan' result in PR comment), 5) CODE REVIEW (team approval), 6) MERGE TO MAIN (merge icon), 7) CD RUNS APPLY (automation, 'terraform apply'), 8) INFRASTRUCTURE CREATED (cloud icons for AWS/Azure/GCP), 9) STATE UPDATED (S3 bucket icon). Add feedback loops: Security scan before Plan, Cost estimate shown in PR comment, Slack notification on successful apply.

Appendix: Quick Reference

Command Cheat Sheet

Command	Description
terraform init	Initialize project, download providers
terraform plan	Preview changes without applying
terraform apply	Apply changes to infrastructure
terraform destroy	Destroy all managed resources
terraform fmt	Format code to canonical style
terraform validate	Validate configuration syntax
terraform show	Show current state (human-readable)
terraform state list	List all resources in state
terraform state show <resource>	Show details of a specific resource
terraform state rm <resource>	Remove resource from state (without destroying)
terraform import <resource> <id>	Import existing resource into state
terraform workspace list	List all workspaces
terraform workspace new <name>	Create new workspace
terraform workspace select <name>	Switch to workspace
terraform output	Show all outputs
terraform output <name>	Show specific output value
terraform force-unlock <id>	Force release of state lock
terraform providers	Show provider requirements
terraform graph	Output dependency graph (dot format)
TF_LOG=DEBUG terraform plan	Run plan with debug logging

HCL Quick Reference

```
# Basic resource
resource "aws_instance" "name" {
  ami           = "ami-..."
  instance_type = "t2.micro"
}

# Variable declaration
```

```
variable "name" {
  type    = string
  default = "value"
}

# Output declaration
output "name" {
  value = resource.type.name.attribute
}

# Local value
locals {
  common_tags = { Environment = var.env }
}

# Data source
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["amazon"]
}

# Module call
module "networking" {
  source = "../modules/networking"
  vpc_cidr = "10.0.0.0/16"
}

# Conditional
value = var.env == "prod" ? "large" : "small"

# For expression
names = [for item in var.list : upper(item)]
```

End of Terraform Textbook | Tech-talks | 2024
For questions and updates, visit tech-talks.io